Evaluating Branching Rollouts for the Gameplay Safety Filter

Christal Chen
Advisor: Professor Jaime Fernández Fisac
ECE 299: Sophomore Independent Work

***Abstract*—Ensuring the safety of autonomous robots in unpredictable environments is critical. The Gameplay Safety Filter utilizes adversarial reinforcement learning to simulate "matches" between a safety controller and a trained disturbance, offering a scalable solution for high-dimensional systems. However, its guarantees rely on the trained disturbance capturing the worst-case scenario. If the true worst-case disturbance does not align, the filter may erroneously permit unsafe actions. This work proposes a runtime branching rollout framework that creates branched trajectories at regular intervals and simulates gameplay with maximally deviated initial disturbance vectors. Results demonstrate that while branching increased filter conservativeness, it successfully uncovered scenarios where the standard trained disturbance failed to predict a potential failure, indicating that branching is a viable tool for increasing the robustness of the gameplay filter.**

***Index Terms*—Autonomous Robotics, Safety Filters, Gameplay Filter, Branching Rollouts**

I. Introduction

As autonomous robots become increasingly integrated into human lives, ensuring the safety of these systems is imperative. Unlike robots operating in isolation, modern autonomous systems must interact with humans in unknown environments characterized by environmental disturbances and unpredictable, uncontrolled agents. To ensure that autonomous systems maintain performance while avoiding catastrophic failure, safety filters are automatic processes that separate task performance and system safety, allowing candidate control actions to pass through only if the filters deem them as safe [1].

One class of safety filters is the Gameplay Filter, a predictive safety framework consisting of two phases: first, it uses an offline Iterative Soft Adversarial Actor-Critic for Safety (ISAACS) scheme to train a controller and an adversarial disturbance; second, during real-time deployment, the filter runs gameplay rollouts, which are simulated matches between the trained controller and the disturbance over a finite, H-step horizon [2]. If the disturbance "wins" the rollout, indicating a potential failure, the filter intervenes with the fallback policy $\pi^{fallback}$.

A primary assumption of gameplay filters is that the trained disturbance represents the absolute worst-case scenario. While empirical results show consistent effectiveness, it is not a guaranteed representation of true environmental dynamics. If a real-world disturbance within the operational design domain (a defined safety standard) is more extreme than what was accounted for during training, the filter at runtime may erroneously predict that a state is safe, leading to catastrophic failure.

To address this limitation of trained disturbances, this work explores a novel runtime rollout framework. By creating branched rollouts at regular intervals and simulating gameplay from maximally deviated alternative disturbances, this scheme aims to improve the safety monitor of the gameplay filter. The results revealed potential failures that the standard gameplay filter's trained disturbance missed, indicating branching as a viable pathway for increasing robustness, though further refinement is required.

II. Related Work

A. Safety Filters

As the application of autonomous robotic systems shifts from operating in isolated, controlled environments to environments with increased human interaction and environmental uncertainty, research in robotics safety has explored control frameworks that aim to guarantee that a system will not experience catastrophic failures under conditions established by a given safety framework [1]. Safety filters consist of three primary components: fallback, monitoring, and intervention. This section discusses three safety filter approaches and how the gameplay filter aligns with and builds upon them.

1) Least Restrictive Safety Filter (LRSF): The LRSF monitors the robot's current state and determines if a candidate control action will drive the robot outside the maximal safe set, which is the largest set of states that does not intersect a state in the set of catastrophic failures. If so, the safety value function becomes negative and the LRSF deems the action as unsafe. With a switch-type intervention, the LRSF overrides the task policy π^{task} with the optimal safety policy, a policy that is guaranteed to keep the robot within the maximal safe set.

Both the maximal safe set and the optimal safety policy are encoded by the safety value function, the solution to the reach-avoid game defined by the framework of Hamilton-Jacobi reachability, where the controller aims to avoid the failure set while a disturbance agent aims to push it in.

The LRSF is a robust and optimal safety filter, yet because Hamilton-Jacobi reachability analysis is computationally expensive, it suffers from the "curse of dimensionality" and becomes impractical for systems with more than six state variables [3]. Because the LRSF uses value-based monitoring and intervenes at the boundary of safety, the controller is aggressive in switching between the task policy π^{task} and fallback policy $\pi^{fallback}$, which can lead to erratic motion [1].

2) Control Barrier Functions (CBFs): Unlike the LRSF, instead of finding or approximating the maximal safe set, a CBF finds a smaller subset of the maximal safe set that is forward invariant, meaning the agent's trajectory will never cross the zero-boundary beyond the safe set [4]. The safe set is defined

as the set of states for which a given state function satisfies $h(x) \geq 0$, and as the function approaches the boundary defined by $h(x) = 0$, there always exists a control input that keeps the system safe by restricting the rate of decrease of $h(x)$ to satisfy a specified constraint [4]. As a result, CBFs address the LRSF's limitation of experiencing control chatter close to the boundary of the safe set. The greatest limitation of CBFs is that they do not have general constructive mechanisms, requiring designers to manually construct the CBF constraints and thereby limiting scalability [4].

3) Rollout-Based Filters: Model Predictive Shielding: Model Predictive Shielding is a rollout-based approach that monitors safety at each control cycle based on a simulated rollout of a dynamics model, where the monitor determines whether or not the safety fallback policy $\pi^{fallback}$ will be able to drive the system into a safe set within a given prediction horizon [5]. If the monitor rollout determines that the fallback is successful, it allows the task policy π^{task} to go through; if the rollout results in a catastrophic failure, the filter overrides the task policy π^{task} with the fallback in a binary policy switch [5]. Rollout-based safety filters are more scalable to high-dimensional dynamical systems, especially when combined with deep learning synthesis, which can efficiently approximate solutions to optimal control problems [1]. However, rollout-based safety filters can be overly-conservative if the chosen terminal safe set is small.

B. The Gameplay Filter

The gameplay filter is a rollout-based, predictive safety filter characterized by simulated gameplay between a trained safety-strategy and adversarial disturbance actor [2]. The controller actor and disturbance actor are trained with the Iterative Soft Adversarial Actor-Critic for Safety (ISAACS) scheme.

ISAACS is an adversarial reinforcement learning scheme that approximates the solution to the Hamilton-Jacobi reachability analysis, jointly training a safety control policy and failure-seeking disturbance policy in a zero-sum game during offline synthesis [6]. Therefore, ISAACS combines the scalability of deep learning with an approximation of a robust value function, effectively addressing the limitations of LRSFs and minimally sacrificing robustness. During offline synthesis, the gameplay filter extends ISAACS to approximate a solution for a reach-avoid game between a safety control policy, which acts as the fallback for the safety filter, and the disturbance actor, which approximates the worst-case disturbance or sim-to-real error that the control policy will face [2].

At runtime, the trained fallback and disturbance are used to create a gameplay rollout, which determines whether or not accepting the candidate action will allow the disturbance actor gaining victory over the control actor in the simulated gameplay, which indicates a catastrophic failure. If so, the safety filter intervenes and overrides the task policy π^{task} with the safety fallback policy $\pi^{fallback}$ [2]. Only a single gameplay rollout trajectory is considered because the trained disturbance is assumed to be the worst-case scenario the controller will face.

With ISAACS, the gameplay filter is more scalable than other safety filter approaches such as LRSFs and CBF-based filters for high-dimensional systems like 36-D quadruped robotic systems [2]. However, the limitation of the gameplay filter is that the extent of its safety guarantee relies on the assumption that the trained disturbance is the true worst-case scenario. If this is not the case, the gameplay filter may erroneously believe that the controller is safe by winning the gameplay rollout, when, in reality, there exists an alternate disturbance that can steer the controller to catastrophic failure.

C. Branching in Model Predictive Control

Outside of ISAACS and the gameplay filter, many model predictive control frameworks have implemented the idea of branching trajectories.

1) Statistical Model Predictive Shielding (SMPS): Similar to MPS, SMPS simulates a learned policy for one step, then simulates a rollout of a backup policy to see if the robot can stay within the safe set; if not, SMPS overrides the learned policy with the backup policy. What makes SMPS different from MPS is that instead of a single rollout with a deterministic disturbance, SMPS simulates multiple rollouts with a sampled disturbance from a probability distribution [7]. Only when the learned policy is safe in all trajectories will the filter allow it to go through.

2) Particle Model Predictive Control (PMPC): Similar to SMPS, PMPC runs multiple rollouts based on sampled disturbances in the environment. What differentiates PMPC from SMPS is its notion of consensus, defined as the extent to which different trajectories share actions. PMPC introduces the consensus horizon, a hyperparameter denoted by N_c. For the first N_c steps of each branch, the sequence of actions is identical so that the policy can be evaluated for safety under different disturbances; after N_c, the actions are allowed to diverge [8]. This mitigates the over-conservativeness of full-horizon consensus, which filters out action sequences that fail to maintain safety under all disturbances, as well as the over-optimism of one-step consensus, which can overfit to a particular disturbance along a trajectory [8].

3) Branch Model Predictive Control (Branch MPC): Branch MPC is a framework developed specifically for reactive motion planning when autonomous agents are in the presence of uncontrolled agents, which is comparable to the disturbance actor in the gameplay filter. The uncontrolled agent's behavior is approximated with a finite set of policies to build a scenario tree (the possible future behaviors of the uncontrolled agent, with its current state as the root and branches as the sequence of actions under a policy). Then, a trajectory tree is built, matching the scenario tree's topological mapping with branches representing the ego agent's responses to each potential policy of the uncontrolled agent [9].

This project applies the branching technique to mitigate uncertainty in the gameplay filter's trained disturbance, similar to how the model predictive control frameworks use multi-trajectory planning to address stochastic disturbances and

uncontrolled agents. This project will also use a consensus-like mechanism to control the duration of initial conditions in each branch, though instead of the control policy, this project will adjust the disturbance in each trajectory.

III. Methodology

This section defines some key variables and terminology for the gameplay filter.

A. The Branching Rollout Algorithm

We consider a dynamical system with state $x \in \mathcal{X}$, control input $u \in \mathcal{U}$, and disturbance $d \in \mathcal{D}$, evolving according to $x_{k+1} = f(x_k, u_k, d_k)$. The failure set is denoted by $\mathcal{F}$, and safety requires $x_k \notin \mathcal{F}$ for all $k \geq 0$.

Every k_b time steps, in addition to the rollout with the trained disturbance, a new branched rollout is created where the initial force vector is negated, which means each component of the disturbance vector is flipped about the origin located at the robot's torso. Since the "worst-case" scenario of the trained disturbance may not account for the true worst-case scenario in the environment, exploring these "opposite" disturbances allows us to probe the limits of the system's safety. This approach is loosely informed by bang-bang control theory, which suggests that the most critical or optimal actions often exist at the boundaries of the control space.

After creating a branch from the original rollout, freeze the negated disturbance for k_f imaginary steps out of the rollout horizon. This ensures that the branch meaningfully deviates from the original rollout. Without this freeze period, the state trajectories might remain too similar to the original rollout, preventing the branch from exploring distinct regions of the state space. By forcing this deviation, we can better evaluate whether the safety fallback policy $\pi^{fallback}$ can maintain the system within a safe set under varied adversarial conditions.

If either the original or the branched gameplay rollout predicts a failure, or defeat of the fallback, the modified gameplay safety filter will intervene with the safety fallback policy $\pi^{fallback}$.

B. Performance metrics

1) Safe rate: The number of failure-free episodes out of total episodes. This metric defines the most important goal of safety filters: whether or not the robot is able to avoid entering a state of catastrophic failure at all times.

2) Task performance: The distance traveled under the walking task policy π^{task}. This metric provides insight into the conservativeness of the safety filter and whether or not the safety filter needed to hinder the robot's operation to ensure safety.

3) Filter frequency: The number of steps when the fallback was selected out of total steps per episode. This metric records how often the safety filter needed to override the task policy π^{task} with the safety fallback policy $\pi^{fallback}$. A high filter frequency can either indicate a more dangerous environment and/or greater conservativeness.

C. Experimental parameters

This section outlines the rationale for the hyperparameter values selected for the trials.

1) Horizon length H: Horizons lengths of 50, 63, and 75 were selected based on preliminary baseline simulations without branching. These values provided a representative range of performance: when $H = 50$, the gameplay filter without branching demonstrated a lower safe rate at 51%, whereas a horizon of 75 yielded a higher safety performance at 95%. $H = 63$ was added retrospectively to examine a horizon length in between the previously chosen values.

2) Branching interval k_b: The branching interval k_b ranged from 1 (branching at every timestep) to 100. Because each episode is capped at 100 steps due to computational time constraints, a k_b value of 100 effectively results in no branching. While a longer episode length (e.g., 1000 steps) would be ideal for calculating more accurate metrics, an episode length of 100 was sufficient to identify clear performance patterns within the available timeframe.

3) Freeze duration k_f: While k_f could theoretically match the full horizon length, freezing a disturbance for the entire duration creates a constant, singular force. Such a scenario is unlikely to represent a true "worst-case" dynamic, as a trained adversarial disturbance would actively adjust to destabilize the robot. Therefore, k_f was limited to a maximum of 30% of the horizon length for this experiment. This heuristic ensures the branched rollout deviates enough to be meaningful without becoming a static, unrealistic simulation.

D. Technical specifications

All simulations were conducted using the MuJoCo physics engine, specifically utilizing the Unitree Go2 quadruped model with a walking task policy π^{task}. Computational workloads were processed on Princeton's Della computing cluster.

IV. Results and Analysis

A. The impact of k_b on performance metrics

In Fig. 1(a), where the baseline (non-branching rollouts) safe rates are plotted as dotted lines, k_b is shown to have little to no impact on the safe rate compared to the horizon length. As the horizon length increases, the distance between the baseline safe rate and the branching rollout safe rates grows, with the branching rollouts experiencing a lower safe rate overall. Since the rollouts with the original disturbance should have precluded any unsafe action, one might expect branching to maintain or improve the safe rate.

While unexpected, this trend of branching rollouts having lower safe rates for long rollout horizons can be explained by control chatter caused by ubiquitously higher filter frequencies for all horizon lengths, as seen in Fig. 1(c). The general trend of this plot indicates that a small k_b (i.e., more frequent branches) increased the filter frequency, as well as the task performance, which starts lower than the baseline task performance but eventually grows to be consistently higher than the baseline for all horizons as seen in Fig. 1(b). Together, Fig. 1(b) and Fig. 1(c) demonstrate an expected trend, which is

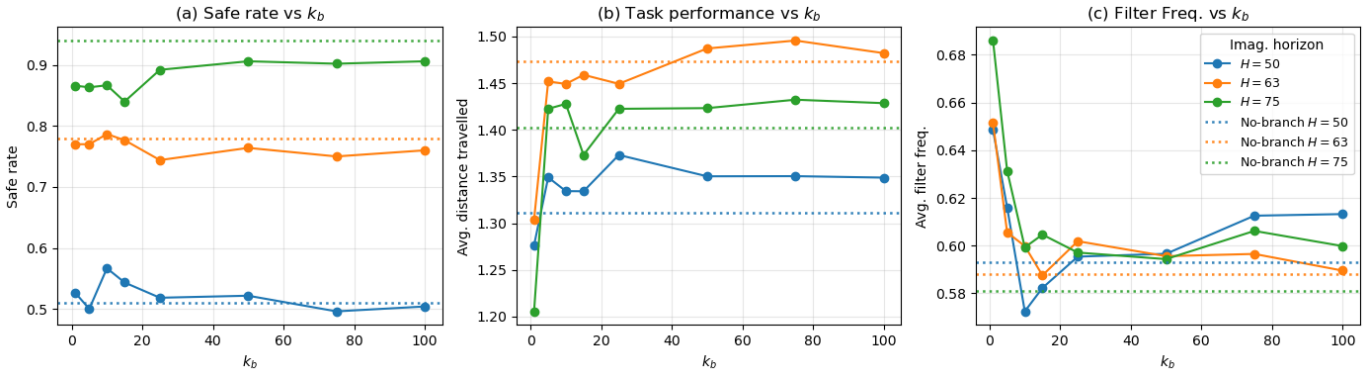

Fig. 1. Performance metrics as a function of k_b, averaged across all values of k_f. (a) Safe rate. (b) Task performance. (c) Filter frequency.

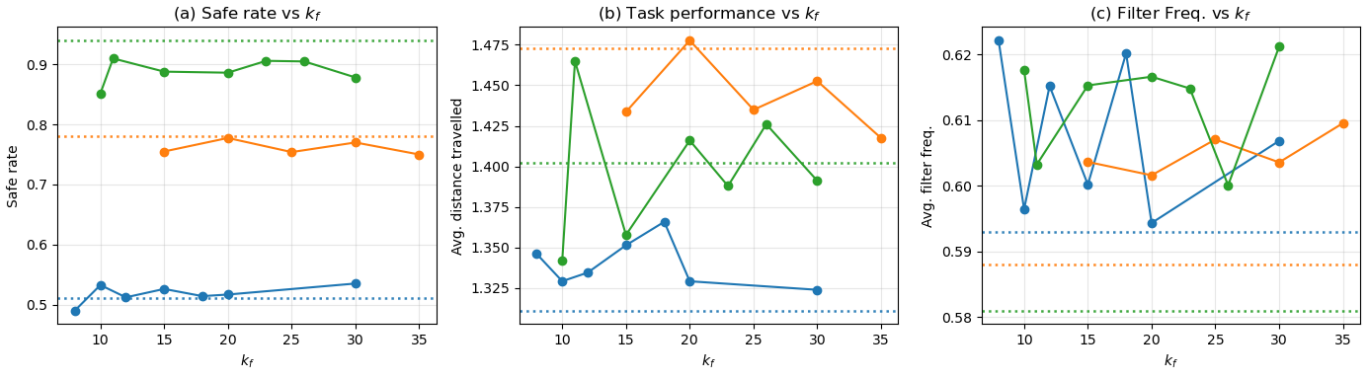

Fig. 2. Performance metrics as a function of k_f, averaged across all values of k_b. (a) Safe rate. (b) Task performance. (c) Filter frequency.

an increased amount of interventions from the fallback policy $\pi^{fallback}$ as the additional branches contribute to the total number of rollout failures that preclude the candidate control actions.

B. The impact of k_f on performance metrics

In Fig. 2(a), a similar trend could be observed as in Fig.1(a) where the hyperparameter had little effect on the safe rate of the filter and rather decreased it for longer rollout horizons. As for task performance in Fig. 2(b) and filter frequency in Fig. 2(c), there is no observable trend, indicating that in this experiment, k_f did not have a significant influence on either performance metric, although Fig. 2(c) reinforces the observation from Fig. 1(c) that branching ubiquitously increased the filter frequency. However, aside from $H = 63$ at high values of k_f, generally, as k_f increased, so did the proportion of branch-only defeats, defined as the proportion of branched rollouts where only the rollout associated with the negated disturbance predicted a failure. As seen in Fig. 3, when the rollout branches deviated more from each other, there was a higher likelihood of the gameplay filter discovering alternative disturbances that led to failure. Fig. 4 demonstrates an example of a branch-only defeat, providing a visualization of the branching rollout scheme achieving its intended function

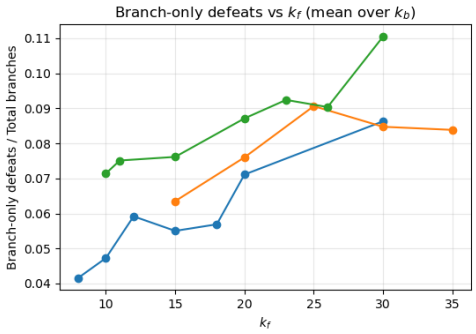

Fig. 3. Rate of branch-only defeats (the fraction of branch events where only the additional rollout predicted a failure out of the total number of branchings) as a function of k_f.

of acting as a secondary verification of the candidate action's safety.

V. Conclusion and Future Work

The finding that branching rollouts generally performed similarly to single-trajectory rollouts is actually an encouraging result, as it reinforces the gameplay filter's reliability in maintaining zero-shot safety.

However, while the non-branching gameplay filter is already robust, it is not perfect. The branching rollout approach

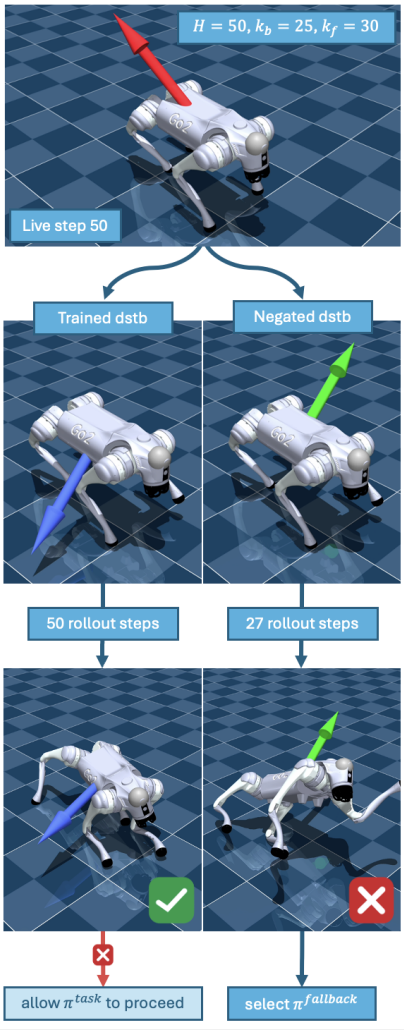

Fig. 4. A visualization of a live step followed by a branching rollout, where the rollout with the original trained disturbance does not fail but the rollout with the negated disturbance does, ultimately causing the safety filter to select the fallback policy $\pi^{fallback}$.

successfully identified specific adversarial disturbances that the offline-trained disturbance missed, some of which led to catastrophic failure. This highlights a critical limitation: the gameplay filter may erroneously believe a state is safe because its trained disturbance is not raising the true worst-case scenario. Ultimately, these results show that branching is a beneficial diagnostic tool for uncovering edge-case failures. To achieve more robust safety guarantees, future iterations of the filter can incorporate branching rollouts to detect and mitigate these hidden risks.

Future research should focus on branching at more meaningful time steps, such as when high uncertainty is detected in the disturbance model, and developing a formal model to define k_f so that branches are guaranteed to be maximally divergent. Additionally, expanding the system to create multiple branches at once rather than just two, and ultimately deploying and testing the filter with branching rollouts on a physical Go2 robot, would provide deeper insights into the framework's real-world efficacy. While this study was limited in scope and time, exploring more diverse branching strategies could further bridge the gap between empirical safety and guaranteed safety.

Acknowledgment

I would like to express my gratitude to Professor Jaime Fisac and Duy Nguyen for their invaluable support and guidance.

References

[1] K.-C. Hsu, H. Hu, and J. F. Fisac, "The Safety Filter: A Unified View of Safety-Critical Control in Autonomous Systems," Annu. Rev. Control Robot. Auton. Syst., vol. 7, no. Volume 7, 2024, pp. 47–72, Jul. 2024, doi: 10.1146/annurev-control-071723-102940.

[2] D. P. Nguyen, K.-C. Hsu, W. Yu, J. Tan, and J. F. Fisac, "Gameplay Filters: Robust Zero-Shot Safety through Adversarial Imagination," Jan. 16, 2025, arXiv: arXiv:2405.00846. doi: 10.48550/arXiv.2405.00846.

[3] S. Bansal, M. Chen, S. Herbert, and C. J. Tomlin, "Hamilton-Jacobi Reachability: A Brief Overview and Recent Advances," Sep. 21, 2017, arXiv: arXiv:1709.07523. doi: 10.48550/arXiv.1709.07523.

[4] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control Barrier Functions: Theory and Applications," Mar. 27, 2019, arXiv: arXiv:1903.11199. doi: 10.48550/arXiv.1903.11199.

[5] O. Bastani, "Safe Reinforcement Learning with Nonlinear Dynamics via Model Predictive Shielding," Oct. 21, 2020, arXiv: arXiv:1905.10691. doi: 10.48550/arXiv.1905.10691.

[6] K.-C. Hsu, D. P. Nguyen, and J. F. Fisac, "ISAACS: Iterative Soft Adversarial Actor-Critic for Safety," arXiv.org. Accessed: Apr. 28, 2026. [Online]. Available: https://arxiv.org/abs/2212.03228v3

[7] O. Bastani and S. Li, "Safe Reinforcement Learning via Statistical Model Predictive Shielding," in Robotics: Science and Systems XVII, Robotics: Science and Systems Foundation, Jul. 2021. doi: 10.15607/RSS.2021.XVII.026.

[8] R. Dyro, J. Harrison, A. Sharma, and M. Pavone, "Particle MPC for Uncertain and Learning-Based Control," Sep. 13, 2021, arXiv: arXiv:2104.02213. doi: 10.48550/arXiv.2104.02213.

[9] Y. Chen, U. Rosolia, W. Ubellacker, N. Csomay-Shanklin, and A. D. Ames, "Interactive Multi-Modal Motion Planning With Branch Model Predictive Control," IEEE Robot. Autom. Lett., vol. 7, no. 2, pp. 5365–5372, Apr. 2022, doi: 10.1109/LRA.2022.3156648.

Appendix

TABLE I
EXPERIMENTAL PARAMETERS FOR BRANCHING INTERVAL (k_b) AND FREEZE DURATION (k_f) ACROSS DIFFERENT HORIZON LENGTHS (H)

H=50		**H=63**		**H=75**	
k_b	k_f **(Freeze)**	k_b	k_f **(Freeze)**	k_b	k_f **(Freeze)**
1	8, 10, 12, 15, 18, 30	**1**	15, 20, 25, 30, 35	**1**	10, 15, 20, 23, 30
5	10, 20, 30	**5**	15, 20, 30	**5**	10, 20, 30
10	10, 20, 30	**10**	15, 20, 30	**10**	10, 20, 30
15	10, 20, 30	**15**	15, 20, 30	**15**	10, 20, 30
25	8, 10, 12, 15, 18, 30	**25**	15, 20, 25, 30, 35	**25**	10, 15, 20, 23, 30
50	8, 10, 12, 15, 18, 30	**50**	15, 20, 25, 30, 35	**50**	10, 15, 20, 23, 30
75	8, 10, 12, 15, 18	**75**	15, 20, 25, 30, 35	**75**	11, 15, 20, 23, 28
100	8, 10, 12, 15, 18	**100**	15, 20, 25, 30, 35	**100**	11, 15, 20, 23, 28